Linux Troubleshooting — Scenario #3: Server Is Slow (High CPU)

DevOps on-call runbook • Syed Aftab • Ubuntu (WSL) • Reference for real incidents & interviews

The alert (7:45 AM): web01 is very slow — pages take 10+ seconds, SSH feels laggy. Nothing was deployed.
Root cause found: two copies of a hand-started script, `/tmp/report-gen`, stuck in an infinite loop, each burning one full CPU core. Stopped with `kill` (SIGTERM). No restart, no data lost.

"The server is slow" is one of the most common tickets. Before killing anything you must prove it is CPU (not memory or disk), find the exact process, and understand what it is and who started it.

The methodology (same 6-phase flow as Scenarios #1 and #2)

#	Phase	Commands	Goal
1	Confirm	`uptime`, `nproc`	Is the CPU really overloaded? Compare load with core count
2	Locate	`top`	Which process is using the CPU? Rule out memory and disk
3	Identify	`ps -o ...`, parent PID, `cat`	What is it, when did it start, who started it, will it come back?
4	Fix safely	`kill PID` (SIGTERM first)	Stop it politely; `kill -9` only if that fails
5	Verify	`pgrep`, `top`, `uptime`	Process gone, CPU idle, load falling
6	Prevent	`flock`, `timeout`, CPU limits, monitoring	Stop duplicate runs and runaway jobs from happening again

LAB 1 First setup attempt — and why it failed

Goal: create a fake "stuck job" with a realistic name. First idea: copy the built-in `yes` program (prints y forever, maxing one CPU core) under a new name, then start it twice in the background.

```
cp /usr/bin/yes /tmp/report-gen
nohup /tmp/report-gen &> /dev/null &      # run twice
```

Part	Meaning
`cp`	copy a file
`/usr/bin/yes`	the `yes` program. `/usr/bin` holds most system commands
`/tmp/report-gen`	the copy, with a realistic name. `/tmp` is a temporary folder, cleared on reboot
`nohup`	**no h**ang**up**: keep running even if the terminal closes
`&> /dev/null`	send all output (normal + errors) to `/dev/null`, the system trash can. Bash shortcut for `> /dev/null 2>&1`
`> /dev/null 2>&1`	the long, portable form: > = normal output, 2>&1 = errors (2) go where normal output (1) goes. Works in plain `sh` too (cron, Dockerfiles)
& (at the end)	run in the **background** and give the prompt back. Prints `[job] PID`

Symptom: `uptime` showed `load average: 0.03, 0.13, 0.11` — an idle machine. The numbers did not match the story.

Troubleshooting the failed setup

```
pgrep -a report-gen             # nothing printed, red X = exit code 1 = no match
ls -l /tmp/report-gen           # -rwxr-xr-x ... 11352352 ... /tmp/report-gen
/tmp/report-gen | head -3       # coreutils: unknown program 'report-gen'
```

Command / part	Meaning
`pgrep`	**p**rocess **grep**: search running processes by name
`-a`	show the full command line next to each PID
`ls -l`	long listing: permissions, owner, size, date. `-rwxr-xr-x` = executable file
size `11352352`	about 11 MB — the classic `yes` is ~40 KB. A clue that this `yes` is different
`| head -3`	pass output to `head`, which shows only the first 3 lines (stops an endless stream)
running it in the **foreground**	no `/dev/null` redirect, so the real error is finally visible

Root cause of the lab failure: newer Ubuntu ships a rewritten **coreutils** where one big program plays every basic command (`ls`, `cp`, `yes`...). It picks which command to act as from **the name it was started with**. Renamed to `report-gen`, it did not know that command and quit instantly. Check with `ls -l /usr/bin/yes` (an `->` arrow means it is a link). Normal use of these commands is unaffected.

Real-world lesson: "I started it but nothing is happening" → (1) do the numbers match the story? (2) is the process running? (3) does the file exist? (4) run it in the foreground. `&> /dev/null` **hides the reason a background job died.**

LAB 2 Working setup — a Bash loop script

```
cat > /tmp/report-gen <<'EOF'
#!/bin/bash
while true; do :; done
EOF
chmod +x /tmp/report-gen
ls -l /tmp/report-gen            # 35 bytes, -rwxr-xr-x
```

Part	Meaning
`cat > /tmp/report-gen`	write the following text into the file, replacing the broken copy
`<<'EOF'` ... `EOF`	**here-document**: every line between the two EOF markers goes into the file. Quotes on 'EOF' = no variable expansion
`#!/bin/bash`	**shebang**: run this file with Bash
`while true; do :; done`	loop forever. `:` is a command that does nothing, so the loop just spins → one core at 100%
`chmod +x`	make the file executable

```
nohup /tmp/report-gen &> /dev/null &      # [1] 223731
nohup /tmp/report-gen &> /dev/null &      # [2] 223767
pgrep -a report-gen
  223731 /bin/bash /tmp/report-gen
  223767 /bin/bash /tmp/report-gen
```

Two copies on purpose: one copy only fills one core, and duplicate runs are a very common real problem (a cron job that overlaps itself, a script restarted without checking, an old worker left alive after a deploy).

STEP 1 Confirm — is the CPU really overloaded?

```
uptime
  08:27:19 up 17:15,  1 user,  load average: 0.94, 0.34, 0.16
nproc
  14
```

Part	Meaning
`uptime`	time up, logged-in users, and **load average** over the last **1, 5 and 15 minutes**
`nproc`	**n**umber of **proc**essors (CPU cores)
load average	roughly how many processes want a CPU at the same time
`0.94, 0.34, 0.16`	rising left-to-right (15 → 5 → 1 min): something **just started** using CPU

Load only means something compared with the core count:

Load ÷ cores	Meaning
below 0.7	healthy
around 1.0	fully busy
above 1.0	processes waiting in line → the server feels slow

Here: load ~2 on 14 cores = fine on this laptop. On a typical **2-core cloud VM**, the same 2 stuck jobs = load 2 on 2 cores = every core taken, and nginx, SSH and everything else wait. That is the "slow server".

STEP 2 Locate — top

```
top          # press q to quit
```

```
%Cpu(s): 15.7 us,  1.1 sy,  0.0 ni, 81.4 id,  0.0 wa ...
MiB Mem :  7566.3 total,  5178.4 free ...
MiB Swap:  2048.0 total,  2048.0 free,  0.0 used
   PID USER     ... S  %CPU %MEM    TIME+ COMMAND
223731 syedaft+ ... R 100.0  0.0  1:26.84 report-gen
223767 syedaft+ ... R 100.0  0.0  1:25.04 report-gen
214352 syedaft+ ... S  22.0  4.8  1:48.71 MainThread
```

Column / line	Meaning
`top`	live process view, refreshes every few seconds, sorted by CPU by default
`PID`	process ID — what you act on
`USER`	who owns it (+ = name cut off)
`S`	state: **R** = running (using CPU), **S** = sleeping (idle, waiting)
`%CPU 100.0`	one **full core**. In top 100% = 1 core, so 14 cores can show up to 1400%
`%MEM 0.0`	almost no memory → purely a CPU problem
`TIME+`	total CPU time used so far (still climbing)
`us` / `sy`	% of all cores used by user programs / the kernel. 15.7% ≈ 2 of 14 cores
`id`	% idle
`wa 0.0`	time waiting on **disk** = 0 → not a disk problem
`Mem free`, `Swap 0 used`	plenty of free RAM, no swapping → not a memory problem
`MainThread`	VS Code — normal program, normal work

STEP 3 Identify — what is it, who started it, will it come back?

Rule: never kill a process you don't understand. It could be a database backup or a payment job.

```
ps -o pid,ppid,user,lstart,etime,cmd -p 223731,223767
   PID   PPID USER     STARTED                  ELAPSED CMD
223731 215443 syedaft+ Tue Oct 27 08:26:44 2026    05:10 /bin/bash /tmp/report-gen
223767 215443 syedaft+ Tue Oct 27 08:26:46 2026    05:08 /bin/bash /tmp/report-gen
```

Part	Meaning
`ps`	**p**rocess **s**tatus — a one-time snapshot (top refreshes, ps does not)
`-o`	choose the columns to show
`pid` / `ppid`	process ID / **parent** process ID (who launched it)
`user`	user it runs as
`lstart`	exact start date and time
`etime`	**e**lapsed **time** since it started
`cmd`	full command, including the file path
`-p 223731,223767`	only these PIDs

Findings: started 2 seconds apart, same parent `215443` → the same job launched twice. Running from `/tmp` is a yellow flag — production jobs live in places like `/opt` or `/usr/local/bin`.

```
ps -o pid,user,cmd -p 215443
215443 syedaft+ /bin/bash --init-file /home/syedaftab04/.vscode-server/.../workbench...
```

Parent CMD	Meaning	Will killing it stick?
`bash` / `-bash`	started by hand in a terminal (**our case** — VS Code terminal)	Yes
`cron` / `CRON`	scheduled job	Until the next scheduled run
`systemd` / `/sbin/init`	a service	May restart automatically → stop the service instead

```
cat /tmp/report-gen
#!/bin/bash
while true; do :; done
```

`cat` prints the file. The script is an infinite loop that does no real work → **safe to stop**, nothing lost.

Ticket note: Two copies of /tmp/report-gen, started by hand 2 s apart from a VS Code terminal, each at 100% of a core. Script is an infinite loop doing no work. Not started by cron/systemd. Safe to stop. (On a real team: ask in chat who ran it first.)

STEP 4 Fix safely — kill (SIGTERM first)

```
kill 223731 223767
[1]-  Terminated    nohup /tmp/report-gen &> /dev/null
[2]+  Terminated    nohup /tmp/report-gen &> /dev/null
```

Part	Meaning
`kill`	sends a **signal** (message) to a process — not always deadly despite the name

Part	Meaning
no option	default signal **SIGTERM (15)**: "please shut down" — the program can clean up first
`223731 223767`	the PIDs (several allowed)
`[1]` / `[2]`	your shell's background job numbers. + = most recent job, - = the one before
`Terminated`	the process received SIGTERM and exited

Signal	Command	When
SIGTERM (15)	`kill PID`	**always first** — lets the program close files, connections, lock files
SIGKILL (9)	`kill -9 PID`	only if SIGTERM fails — kernel removes it instantly, no cleanup (risk of half-written files, stale locks)

The shell printed *Terminated* only because it started those jobs. On a real server `kill` prints nothing — you must verify.

STEP 5 Verify — process gone, CPU idle, load falling

```
pgrep -a report-gen     # nothing printed = no process left
top                     # Tasks: 1 running ... 97.9 id; report-gen gone
uptime                  # load average: 0.66, 1.41, 0.92  (falling)
```

Check	Result	Meaning
`pgrep -a report-gen`	no output	both processes are gone
`top` `%Cpu(s)`	`97.9 id, 1 running`	CPU back to idle
`uptime`	`0.66, 1.41, 0.92`	1-min load already below 5-min → falling. It drops gradually because it is an average

Incident resolved. Lab cleanup: `rm /tmp/report-gen`

STEP 6 Prevent — stop runaway and duplicate jobs

Root cause: the same job was started twice and had no limit on how long or how hard it could run.

1. Never let a job run twice at once — `flock`

```
flock -n /tmp/report-gen.lock /opt/scripts/report-gen.sh
```

Part	Meaning
`flock`	run a command while holding a **lock file**
`-n`	**n**on-blocking: if the lock is already held (job still running), exit right away instead of starting a second copy
`/tmp/report-gen.lock`	the lock file — only one process can hold it at a time

2. Give every job a time limit — `timeout`

```
timeout 30m /opt/scripts/report-gen.sh
```

`timeout 30m` sends SIGTERM if the job is still running after 30 minutes, so a stuck loop can't run forever. In cron: `0 2 * * * flock -n /tmp/rg.lock timeout 30m /opt/scripts/report-gen.sh`

3. Cap how much CPU a job can take

```
nice -n 19 /opt/scripts/report-gen.sh                    # lowest priority: web traffic goes first
sudo systemd-run --scope -p CPUQuota=50% /opt/scripts/report-gen.sh   # hard cap: max half of one core
```

Part	Meaning
`nice -n 19`	run at the lowest priority (range -20 highest to 19 lowest). It can still use idle CPU, but yields to everything else
`systemd-run --scope`	run the command inside a temporary systemd unit, so limits can be applied
`-p CPUQuota=50%`	never more than 50% of one core. For services, put `CPUQuota=` in the unit file. In Kubernetes the same idea is CPU **limits**

4. Run scheduled jobs from a proper place — `/opt` or `/usr/local/bin`, owned by a service account, under cron or a systemd timer. Not ad-hoc from `/tmp`.

5. Know before users do — alert when **load ÷ cores** stays above ~0.8 for 5+ minutes, or a single process holds high CPU (Prometheus node_exporter, Datadog, CloudWatch CPUUtilization alarms).

REF Full command reference

Command	What	Why here
`uptime`	load average 1 / 5 / 15 min	confirm CPU pressure and trend
`nproc`	number of CPU cores	load only means something vs. cores
`top`	live processes, sorted by CPU	find the process; rule out memory (Mem/Swap) and disk (wa)
`pgrep -a NAME`	find processes by name	is it running? how many copies?
`ps -o pid,ppid,user,lstart,etime,cmd -p PID`	detailed snapshot	when, by whom, what file
`ps -o pid,user,cmd -p PPID`	inspect the parent	terminal, cron or systemd? will it come back?
`cat FILE`	read the script	understand it before stopping it
`kill PID`	SIGTERM — polite stop	first choice
`kill -9 PID`	SIGKILL — forced	only if SIGTERM fails
`ls -l FILE`	file details	exists? executable? size clue
`CMD | head -3`	first 3 lines only	see output/errors without flooding
`nohup CMD &> /dev/null &`	background, survives hangup, output discarded	how the job was started (and why errors were hidden)
`cat > FILE <<'EOF'`	here-document	write a multi-line file from the shell
`chmod +x FILE`	make executable	so the script can run
`flock -n`, `timeout`, `nice`, `CPUQuota`	locks, time limits, priority, CPU caps	prevention

Interview answer (say this)

"Server is slow? First I check uptime and compare load with nproc — load near or above the core count means CPU pressure. In top I confirm it's CPU, not memory or disk (free RAM, no swap, wa at zero), and find the process using it. Before touching it, I use ps to see when it started, its parent PID — cron, systemd or a person — and what the script does. If it's safe, I send SIGTERM with kill, and only use kill -9 if that fails. I verify with pgrep, top and a falling load average. Then I prevent a repeat: flock so it can't run twice, timeout so it can't run forever, and nice or a CPU quota so it can't starve the server, plus an alert on load per core."